Transformer:

Self-Attention: $\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$; Q: Suchanfrage ; Key: Index ; V: Value , $\sqrt{d_k}$: wird verwendet, damit softmax nicht in die Sättigung geht

↳ Berechnet Ähnlichkeit Q,K-Value und gewichtet Values darauf | **Cross-Attention:** Q kommt aus Decoder, K und V aus Encoder (oder anderen Quelle) → genutzt bei Seq2Seq-Übersetzen

Embeddings: Positional Encoding: $PE(pos, 2i) = \sin(pos/1000^{2i/d})$; $PE(pos, 2i+1) = \cos(pos/1000^{2i/d})$ | Alternativen zu PE → RoPE: Rotation von q_i und k_j | Alternative: ALiBi | **Subword-Tokenisierung:** „Offener Wortschatz bei begrenztem Vokabular":

Byte Pair Encoding (BPE): Häufige Zeichen werden iterativ zusammengeführt | Word Piece: Wählt Subwords nach Modellwahrscheinlichkeit | Unigram LM: startet groß, entfernt unwahrscheinlichste Token

Encoder-/Decoder-only Transformer: **Encoder-only (BERT):** - Bidirectional Attention; - Pretraining: Masked Language Model + Next Sentence Prediction ; Zweck: Sprachrepräsentation

↳ für Understanding Tasks; Typische Tasks: Klassifikation, Similarity, QA, NER

Decoder-Only (GPT): Kausal (Masked) Attention; Pretraining: Next-Token-Prediction; Zweck: Textgenerierung; Typische Tasks: Chatbots, Dialog, Code-Generierung

Finetuning: **In-Context Learning:** [Context window limited] - Zero-Shot: Aufgabe im Prompt ohne Beispiele ; One/Few-shot: Bsp im Prompt

Instruction Finetuning: Paare aus Prompt (Instruction) und Completion ; Gewichte werden angepasst ; → Performance anderer Tasks sinkt (Catastrophic Forgetting)

Full Finetuning vs PEFT: Full Finetuning: Alle Parameter werden Trainiert ; PEFT: Nur wenige Parameter

Parameter Efficient Finetuning (PEFT): LoRA: 1. W_s werden eingefroren, 2. Zerlegung von $W_{q/k/v} = B_{q/k/v} \times A_{q/k/v}$

3. Addieren zu den eingefrorenen Gewichten → Training der Parameter der Matrizen $B_{q/k/v}$ und $A_{q/k/v}$

Bsp:
- $d \times k = 512 \times 64 = 32.768$ Parameter

LoRA mit Rang $r = 8$:
- $\dim(B) = d \times r = 512 \times 8 = 4.096$ Parameter
- $\dim(A) = r \times k = 8 \times 64 = 512$ Parameter
- 4.608/32.768 ≈ 14% der ursprünglichen Modellparameter oder
- 86 % Reduktion der zu trainierenden Parameter

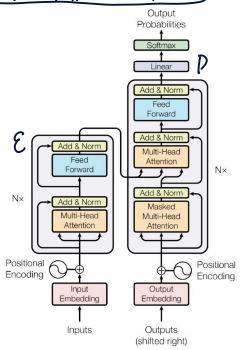
} 4.608 Parameter

QLoRA: 4 Bit Quantization; Prompt Tuning: Fügt trainierbare Softprompts zum Input hinzu, Weights eingefroren, nur Soft Prompt trainiert

Feed Forward Network (FFN):
- Wird auf jedes Token seperat angewandt
- Keine Interaktion zwischen Tokens
- Typisch: 1×Hidden, 4×Input Dimension
- Varianten: - ReLU/GELU (Klassisch)
  - SwiGLU (State of the Art) → besser als ReLU
  - GELU (Gaussian Error Linear Unit)

Parameterberechnung (Bert)

1. Token-Embeddings: $s \cdot d$
2. Transformer-Layer:
   (SA) Self-Attention: $4(d^2 + d)$
   (FFN) FFN: $(d+1) \cdot d_{ff} + (d_{ff}+1) \cdot d$
   L·(SA + FFN)
3. Embeddings + Transformer: $s \cdot d + L \cdot (SA + FFN)$



RLHF: 1. Trainieren des Basis LLM mit Supervised Learning.

2. Reward Modelling: Sammle Paarvergleiche für den selben Prompt → Mensch sagt A oder B besser, → Trainiere Reward Modell $r_\theta(x, y) = (\text{Prompt}, \text{Antwort})$, 3. Policy Optimization: Optimiere Policy um Reward zu max. KL-Penalty damit Policy nicht zu weit abdriftet

Direct Preference Optimization (DPO): Kein separates Reward Modell notwendig, Optimiere LLM auf Präferenzen → Skipte oder Besser

RLHF	vs	DPO
- Trainiert Reward Modell aus Präferenzen		- Direktes supervised Learning auf Präferenzdaten
- Optimiert LLM via RL (z.B. PPO)		- KL-Regularisierung implizit in der Zielfunktion
- Explizite KL-Regularisierung zur Referenzpolicy		- Off-policy, stabil
- On Policy, explorativ - kann (theoretisch) beliebige Modellantworten bewerten.		- Deutlich einfachere Pipeline
- Hoher Engineering/Compute Aufwand		- Hohe Daten und Compute Effizienz
- Anfällig für Reward Hacking		- Trainiert nur auf vorhandenen Antworten
- Theoretisch sehr ausdrucksstark		- → Optimiert Präferenzen lokal um die Referenzpolicy

Reasoning in Sprachmodellen: Chain-of-Thoughts: LLM-Schritt für Schritt denken lassen → Bessere Perform. auf komplex Problemen.

Advanced Reasoning: 1. Tree of Thoughts: Mehrere Gedankenpfade erkunden; 2. Self-Consistency: Mehrere Samples generieren, Mehrheits notwendig

3. Tool use - LLM nutzen externe Tools (Rechner) | 1. Nutzer Query → Embedding, 2. Ähnlichkeitssuche in Vektor Datenbank, 3. Schreibe Datenbank + Query → LLM

Interaktion mit externen Datenbanken: RAG: 1. LLM gewinnt Antwort basierend auf Kontext | Vorteile: Aktuelle Infos, Quellenangaben, Reduziert Halluzination

Darstellungsmöglichkeiten von Text: **Bags of Words (BoW):** „Jedes Wort wird als One-Hot-Vektor repräsentiert."

Formal Definition: Vokabular $V = \{w_1, w_2, \dots, w_{|V|}\}$; One-Hot-Vektor für Wort: w_i: $v_i \in \mathbb{R}^{|V|}$ mit $v_i[j] = 1$ wenn $j = i$, sonst 0

Bsp: Input: "language technologie is awesome" → Vokabular: {awesome: 0, technologie: 1, is: 2, hello: 3, world: 4, language: 5}

Input 2: "Hello Hello World" BoW: $x_1 = [1, 1, 1, 0, 0, 1]$; $x_2 = [0, 0, 0, 2, 1, 0]$

Eigenschaften von BoW: Dimension: |V| oft 30k-100k bei großen Korpora, Sparsity: meisten Einträge = 0, Verlust der Wortreihenfolge | Probleme: Keine Semantik: cos("King", "Queen") = cos("King", "Table") ≈ 0, kein Wortreihenfolge

Hoher Speicherbedarf: |V| = 100k → jeder Vektor benötigt 400KB, Data Sparsity: Seltene Wörter haben wenige Beispiele

TF-IDF: „Häufige Wörter sollten nicht dominieren. TF-IDF gewichtet Wörter nach ihrer Diskriminierungskraft

Formeln: $TF(w,d) = \left(\frac{\#w \text{ in } d}{\#\text{Wörter in } d}\right)$; $idf(w,D) = \log\left(\frac{|D|}{|\{d \in D \mid w \in d\}|}\right)$, Cos Similarity: $\cos(v_1, v_2) = \frac{(v_1 \cdot v_2)}{|v_1| \cdot |v_2|}$ (1 ähnlich, 0 nicht ähnlich)

$TF\text{-}IDF(w,d,D) = TF(w,d) \times IDF(w,D)$ | Anwendung: Dokumentenvergleich: Cosine Similarity zwischen TF-IDF-Vektoren ; Information Retrieval: Ranking von Dokumente nach Relevanz ; Input für ML-Modelle

Word Embeddings: Distributionelle Semantik „you shall know a word by the company it keeps" → ähnliche Wörter → ähnliche Vektoren. Bias in Embeddings: Verstärken geschlechtliche Vorurteile

Word2Vec: CBOW: Richtung: Kontext → Zielwort (gut für häufige Wörter), Skip-gram: Zielwort → Kontext (Besser für seltene Wörter)

FastText: Bag of Character n-grams: bsp: where = <wh, whe, her, ere, re>, <where> → Besser Embeddings für seltene Wörter + Funktion out of Vocabulary

Character Embeddings: CNN über Characters: Filtergröße 3,4,5 über char-Sequenz ; RNN über Characters: LSTM/GRU über char-Sequenz, letzte Hidden State = Word Vector ; Hybrid: Char-CNN + Word-Embeddings

Recurrent Neural Networks: Motiv: Seq. Daten werden viel verwendet und können bei RNN verwendet werden | **Vanilla RNN:** Formeln: $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$

$y_t = g(W_y \cdot h_t + b_y)$ mit h_t: Hidden State zum Zeitpunkt t, x_t: Input zum z.z.t, y_t: Output z.z.t ; Lernbare Parameter: W_x, W_h, b_h, b_y ; Eigenschaften: Variable Inputlänge, Parameter Sharing, Seq. Berechnung

Backpropagation through time: Lossfunktion: $L = \sum_{t=1}^{T} l(y_t, \hat{y}_t)$ mit L: Loss, y_t: Label zum Zeitpunkt t, $\hat{y}_t$: Vorhersage | Jeder Zeitabschnitt kann zu Loss beitragen | RNN-Probleme im Überblick

Prob.	Ursache	Auswirkung	Lösung
Vanishing Grad.	tanh-Ableitung < 1	Keine langen Abhängigkeiten	LSTM, GRU
Exploding Grad.	große W_h-Werte	Numerisch instabil	Gradient Clipping
Inf. Bottleneck	h_t muss alles speichern	Inf. Verlust	Attention
Seq. Berechnung	h_t hängt von h_{t-1} ab	Keine Parallelisierung	Transformer

LSTM: „LSTM hat einen Memory-Mechanismus. → Modell lernt welche Informationen Gespeichert/Gelöscht oder ausgelesen und für das

1. Welche Informationen werden behalten ("forget gate") → $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
2. Welche neuen Inf. hinzugefügt ("input gate") → $i_t = \sigma(W_i [h_{t-1}, x_t] + b_i$; $\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$ (Im zweiten LSTM/GRU Input von rechts.)
3. Update des alten Zustands c_{t-1} → $C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$ | Bi-direc. LSTM/GRU: → Output wird Konkatiniert
4. Bestimmung des Outputs und neuen hidden states → $o_t = \sigma[W_o [h_{t-1}, x_t] + b_o)$, $h_t = o_t \cdot \tanh(C_t)$

Parameter pro Gate: $(x+h) \cdot h + h$ | **LSTM Varianten:** 1. Peephole Conn. → $f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$. Zieht sich durch die anderen Schritte; 2. Schloßpolt Forget und Input-Gates → $C_t = f_t \cdot C_{t-1} + (1 - f_t) \cdot \tilde{C}_t$

GRU: Update gates ersetzt sep. Forget- und Input-Gates → Schneller Trainierbar, weniger Parameter, Formeln: $z_t = \sigma(W_z [h_{t-1}, x_t]$ (Update Gate), $r_t = \sigma(W_r \cdot [h_{t-1}, x_t]$ (Reset Gate), $\tilde{h}_t = \tanh(W \cdot [r_t \cdot h_{t-1}, x])$, $h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot \tilde{h}_t$

Seq2Seq u. Enc/Decoder Architektur: „PE → EN, Text-Long → Kurz, Dialog: Input → Response, Bildbeschreibung: Bild → Text" | **Encoder:** Verarbeitet Input variabler Länge und transf. ihn in Vektor def. Größe → Inf. steckt nun in Hidden State Vektor → dient als "prev. hidden state" h_{t-1} des Decoders | **Decoder:** muss Sequenz var. Länge vorhersagen → Basiert auf hidden state Vektor des Encoders, → Input ist jeweils das zuvor generierte Wort des Decoders

Beam Search: 1. Starte mit k-Beam Size leeren Sequenzen, 2. Für jeden Zeitschritt: 2.1. Expandiere alle k-Beams mit allen möglichen nächsten Wörtern, 2.2 Berechne Score für jede partielle Seq. 2.3. Behalte nur die top-k Seq.

3. Am Ende: Wähle Sequenz mit höchstem Gesamt-Score. → zu jeden Zeitschritt der gesamten Input nicht und die Relevanz für den nächsten Zeitschritt bewertet. | Umsetzung: Anstelle des letzten „hidden state" des Encoders, bekommt

Bahdanau-Attention: „Decoder wird so erweitert, dass der Decoder Zugriff auf alle hidden states der Encoders. → zu jeden Zeitschritt entscheidet der Decoder, wie relevant ein bestimmter „hidden state" ist

Umsetzung: Mit Hilfe der Gewichte $a_{t,i}$, kann der Decoder lernen, welche Teile des Inputs zu einem gegebenen Zeitpunkt, also für die Vorhersage des nächsten Tokens, am wichtigsten ist. Der Decoder kann sozusagen eine Aufmerksamkeit ("Attention") auf die relevanten Teile des Inputs lernen.